

DEDICATED TO INNOVATION

www.bacancy.com





**Customer Satisfaction
Is Our Highest Priority**



Employee Matters



In this session

What we are learning in this session

- ▶ Asynchronous Programming
- ▶ Multithreading and Parallel Programming

Let's Start

Asynchronous Programming

- ▶ **Asynchronous Programming**
 - async and await keywords
 - Task-based asynchronous programming
 - Parallel processing with Task and Task.Run
- ▶ **Multithreading and Parallel Programming**
 - Using Thread, Task, and Parallel classes
 - Synchronization mechanisms
 - Concurrent collections
 -
 - **Key Features:** Non-blocking I/O operations
 - Efficient resource utilization

Asynchronous Programming

- **async**: Marks a method as asynchronous, enabling the use of await.
- **await**: Suspends the method execution until the awaited task is complete.

Example

```
public async Task FetchDataAsync()
{
    var data = await GetDataFromAPI();
    Console.WriteLine(data);
}
```

Asynchronous Programming

Task Class: Represents an asynchronous operation.

Advantages

- Simplified handling of asynchronous code.
- Supports cancellation and continuations.

Example

```
Task<int> task = Task.Run(() => ComputeSum());  
int result = await task;
```

Multithreading and Parallel Programming

► Definition

Running multiple threads in parallel within the same process.

Key Objectives

- Utilize multiple CPU cores.
- Improve performance for computationally intensive operations.

Example

```
Thread t = new Thread(() =>  
    Console.WriteLine("Thread Example"));  
t.Start();
```

Multithreading and Parallel Programming

► Synchronization Mechanisms

Used to avoid data corruption in shared resources.

Mechanisms in C#

- lock: Simplest way to synchronize access.
- Monitor: Fine-grained control.
- Mutex: Inter-process synchronization.
- Semaphore: Controls access to a resource pool.

```
lock(obj)
{
    sharedResource++;
}
```

Thank you